

Programming project 3 – Bicomplex numbers

Background

The bicomplex numbers $\mathbb{B}$ are an extension of the complex numbers $\mathbb{C}$, using *three* imaginary units i , j , and k . A bicomplex number $z \in \mathbb{B}$ is an expression of the form

$$z = a + bi + cj + dk \quad \text{with } a, b, c, d \in \mathbb{R}. \quad (1)$$

Addition and subtraction is explained on $\mathbb{B}$ like on complex numbers (i.e., as polynomials in i, j, k), and for multiplication, the relation $i^2 = -1$ is amended by the following ones:

$$i^2 = -1, \quad j^2 = -1, \quad k^2 = +1, \quad ij = ji = k, \quad ik = ki = -j, \quad jk = kj = -i. \quad (2)$$

Thankfully, the usual computation rules (associative, commutative and distributive laws) hold in $\mathbb{B}$. The complex numbers can be regarded as a subset of $\mathbb{B}$, namely, by setting $c = d = 0$, and the real numbers as $b = c = d = 0$. Thus addition and multiplication of real and complex numbers with bicomplex numbers are also defined.

We further define the *conjugate* of $z \in \mathbb{B}$ as

$$z^\dagger = a + bi - cj - dk, \quad (3)$$

its absolute value as

$$|z| = \sqrt{a^2 + b^2 + c^2 + d^2}, \quad (4)$$

and the so called *square complex norm* ν as

$$\nu(z) = zz^\dagger \in \mathbb{C}. \quad (5)$$

With the help of these, one can define the multiplicative inverse of z as

$$z^{-1} := \nu(z)^{-1} z^\dagger, \quad (6)$$

and then indeed $zz^{-1} = 1$. We denote $\frac{z}{w} = zw^{-1}$ as usual. Note however that not every $z \in \mathbb{B} \setminus \{0\}$ is invertible; namely, z is invertible if and only if $\nu(z) \neq 0$. In numerical computation with floating point numbers, it is impossible to determine reliably whether $\nu(z)$ is zero; therefore, we will consider z as invertible if $|\nu(z)| \geq \epsilon |z|^2$ where (somewhat arbitrarily) $\epsilon = 10^{-10}$.

We will also consider functions $f : \mathbb{B} \rightarrow \mathbb{B}$. For example, polynomials (with bicomplex coefficients and for bicomplex arguments) can be defined as usual; and the exponential function is, for $z = z_1 + z_2 j$ with $z_1, z_2 \in \mathbb{C}$, defined as

$$\exp(z) := \exp(z_1) (\cos(z_2) + j \sin(z_2)). \quad (7)$$

The zeros of such functions, which we will compute numerically, have some surprising features. In particular, equations may have more solutions in $\mathbb{B}$ than in $\mathbb{C}$. For example, the polynomial $P(z) = z^n - 1$ has n zeros in $\mathbb{C}$, but n^2 zeros in $\mathbb{B}$.

The above should contain all essential facts about $\mathbb{B}$; more information can be found in the literature, for example here, or search for “bicomplex number” or “multicomplex number”. But note that conventions for the naming of i, j, k etc. can vary.

Tasks

As a general convention, all real numbers in the following will be represented by `double` floating point numbers in the code, and all integers by the `int` type. Complex numbers are represented by the `ComplexNumber` type (this class, developed in the lectures, is included in the code template). Bicomplex numbers are represented by the class `BicomplexNumber` which will be developed in Part 1 below.

In object types, all (non-static) methods specified on this assignment sheet should be declared as `public`, fields and internally used helper methods (if any) should be declared as `private`. Functions and procedures (static methods) should be left with their default visibility.

In computations where fractions or divisions occur, cases where the denominators are zero should be ignored (i.e. the relevant parameter should be assumed to be non-zero).

Unless specified otherwise, you do not need to check input parameters for consistency or for exceptional input values.

1) A class for bicomplex numbers

In this part, we will create a class `BicomplexNumber` to represent an element of $\mathbb{B}$. As in the case of `ComplexNumber`, these objects should be immutable. An empty class `BicomplexNumber` is given. Please add the following to it:

- (a) Implement two constructors of the class. The first one should take four real numbers as arguments (a, b, c, d) and create the bicomplex number $a + bi + cj + dk$. The other should take two complex numbers (z_1 and z_2) as arguments, and create the bicomplex number $z_1 + z_2j$. Write methods `getA`, `getB`, `getC`, `getD` (no parameters) that retrieve the values of a, b, c, d from the object. *Warning:* Please test these constructors and methods very carefully, as the assessed tests for the following parts all rely on them.
- (b) Implement two methods `abs` and `squareComplexNorm` that return $|z|$ and $\nu(z)$, respectively, where z is the “current” bicomplex number. They should have no arguments, and their return types should be `double` and `ComplexNumber`, respectively.
- (c) Implement two methods `negate` and `conjugate` (no arguments) that return $-z$ and $z^\dagger$, respectively.
- (d) Implement methods `add`, `subtract`, and `multiply` which all take a bicomplex number w as their argument and return $z + w$, $z - w$ and zw respectively.
- (e) Implement a method `invert` (no argument) and `divide` (one bicomplex number, w , as argument) which return z^{-1} and z/w , respectively. They should throw a `java.lang.ArithmeticException` if the number z , respectively w , is not invertible.
- (f) Override the `toString` method of the class, so that it returns the value of the bicomplex number as in the following example:

$$3.05 - 5.1 i + 0.0 j + 4.542 k$$
(Whitespace outside numbers is not relevant. All other characters should be as in the example.)
- (g) Implement a method `exp` (no arguments) which returns $\exp(z)$.

2) Bicomplex functions

We now consider functions $f : \mathbb{B} \rightarrow \mathbb{B}$. Create a class `BicomplexFunction` to represent such a function f . The class should have only one method, `valueAt`, with one argument of type `BicomplexNumber` and the same return type, which should return the value of f at the given point. The return value of the method does not matter; the idea is (as with `RealFunction` in the lectures) that subclasses of `BicomplexFunction` will provide the actual function values for concrete functions f by overriding the method `valueAt`.

As an example of this principle, then create a subclass `BicomplexPolynomial` of `BicomplexFunction` to represent a polynomial $P(z) = \sum_{j < n} c_j z^j$ with coefficients $c_j \in \mathbb{B}$. Its constructor should take as arguments the coefficients c_j as an array of `BicomplexNumber` objects.

3) Finding zeros

In this part, we aim to find the zeros of functions $f : \mathbb{B} \rightarrow \mathbb{B}$ using the secant method. The secant method is a root finding algorithm somewhat similar to Newton's method: Given two start values $x_0 \neq x_1$, which are close enough to a zero of the function f , one computes the sequence

$$x_n = x_{n-1} - f(x_{n-1}) \frac{x_{n-1} - x_{n-2}}{f(x_{n-1}) - f(x_{n-2})}, \quad n \geq 2. \quad (8)$$

One keeps computing $x_2, x_3, \dots$ until one finds an n such that $|x_n - x_{n-1}| < \delta$ (we take $\delta = 10^{-10}$). Then, x_n is (under suitable conditions) a good approximation to a zero of f .

The following code should go into a class `ZeroFinder`, to be created.

- Implement a function (static method) `secantMethod` which takes as arguments f (a `BicomplexFunction`) and two numbers $x_0, x_1 \in \mathbb{B}$; it applies the secant method to this function and start values, and returns the approximation of a zero of f . *Important:* Convergence of the secant method can sometimes fail for two main reasons. It may be that the sequence does not stabilize, i.e., $|x_n - x_{n-1}| < \delta$ is not reached within a reasonable number of steps (say, $n = 50$). It can also happen that this stopping condition is reached, but still $f(x_n)$ is not near zero (which can occur due to roundoff errors and the peculiar nature of the multiplication in $\mathbb{B}$). In both cases, your code should throw a `java.lang.ArithmeticException`.
- Write a function `findAllZeros` which takes f (a `BicomplexFunction`) and a floating point number $r > 0$ as parameters. It should find *all* zeros of the function f , assuming that these are in the region $|z| < r$. This should be achieved by choosing various suitable start values x_0, x_1 and running the secant method for each of these. It is up to you how to pick the start values, but for the program not to run too long,* it is not advisable to choose more than 10,000 different pairs of start values. The output of the function should be of type `List<BicomplexNumber>`. The list should not contain duplicates, including values that are equal up to roundoff/approximation errors. For full marks, make full use of the relevant features of the Java collections framework. (This may even involve reading a bit further than what was discussed in the lectures. Hint: you can override Java collections check for equality of elements by considering $z_1 = z_2$ if $|z_1 - z_2| \leq \epsilon$, with $z_1, z_2 \in \mathbb{B}$).
- Using the previous parts as appropriate, write a function `findSolution` that, given $c \in \mathbb{B}$ as a parameter, finds a solution $z \in \mathbb{B}$ of the equation $\exp(z) = cz$, and returns this z as a `BicomplexNumber`. The equation may have several solutions; it is not relevant *which* of these your function returns.

*The assessed tests will time out after 60 seconds.

4) Documentation

(a) *Source code documentation:*

Within the source code, add Javadoc comments to every (non-private) function, procedure, method and class. In these, describe the purpose of the function, class, etc., and document all parameters and return values. Also, document all exceptions that are thrown by your own code. Generate the Javadoc output and hand it in together with the project.

(b) *Test plan:*

For the methods `multiply`, `invert` and `divide` in Part 1(d)–1(e), write a test plan: What relevant examples can you use to test your code? What are the input values you would use, and what are the expected outputs? Give at least 4 relevant test cases for each of the methods, covering different scenarios including “edge cases” and exceptional values where applicable.

How to prepare your submission

Start by downloading the template files from Moodle. Some of the Java classes that you will work with are already defined there.

Read the assignment sheet carefully. Be sure to implement all functions and methods with exactly the names, parameters and return types that are specified, and with parameters in the order as given.

While preparing your code, please follow these style and formatting guidelines. (This forms part of the assessment.)

- Place all code block delimiters – { and } – on separate lines, aligned on the same column.
- Within code blocks, indent the code by 4 spaces with respect to the surrounding code.
- Choose all names for variables, parameters, and functions to start with a lowercase character.

Make sure that your code compiles correctly. If the source code files that you submit do not compile (for whatever reason), you will receive zero marks for the affected parts of the project.

The code template also includes a unit test. This test does *not* check the output of your code – it only verifies whether you have declared your functions with the correct names and with the correct parameter/return types. You are advised to use the unit test to check your function declarations for any typos. (You may need to create all classes described on the assignment sheet before running the test, even if you aim only at a partial solution.)

Test every piece of your code with meaningful test data, including any special or exceptional values. Verify that the output of your functions is plausible.

Once you are satisfied with the code, create a JAR file in BlueJ as follows:

- Select “Project → Create Jar file” in the menu.
- Leave “Main class” set to “none”.
- ***Be sure to tick “Include source”*** and “Include BlueJ project files”.
- Click “Continue” and save the file in a convenient location.

This JAR file is the one that you need to submit. In addition, please create a PDF file with your test plan.

Mark scheme

Your submission will be marked as follows. Partial solutions are acceptable, and will be awarded partial marks.

Code correctness (24 marks) Your code will be tested by an automated process (a collection of 48 unit tests) to check whether it works correctly for various values of the input parameters. For each of these test cases that you pass, 0.5 marks will be awarded. Marks are awarded entirely on the criterion whether the output values of your code match the conditions specified on this assignment sheet.

Coding style (12 marks) Your source code will be read by the examiner. Marks will be awarded to reflect whether your code is readable, is appropriately structured, and follows the coding style conventions mentioned above. Marks may be subtracted for code that is functional but is implemented in a non-transparent or overly complicated way.

Source code documentation (5 marks) These marks are awarded for appropriate Javadoc comments on the functions in your code (see part 4).

Test plan (6 marks) These marks are for the test plan as described in part 4.

In total, 47 marks are available. Your raw mark out of 47 will be moderated and scaled to the University scale 0–100.